DATA SCIENCE & ANALYTICS · SESSION 11

Regression — OLS, Ridge & Lasso

Predicting a number, reading the coefficients, and taming overfitting

Asst.Prof.Dr.Chaiwat N. · 2026

2026 · Bachelor Program · 3 hrs · Workshop Format · Hands-on Coding

Session Agenda

01 **The Prediction Problem**
Predicting store revenue

02 **Linear Regression**
The line of best fit and its coefficients

03 **Reading the Model**
R^2, residuals, multicollinearity

04 **Why Regularization**
Overfitting and the bias–variance trade-off

05 **Ridge & Lasso**
Shrinking coefficients and selecting features

06 **Workshop Block**
OLS vs Ridge on your data — report the gain

01

PART 01

The Prediction Problem

Predict monthly store revenue from store characteristics and marketing spend — a regression problem where the output is a number, not a category.

Regression predicts a number — and tells you which inputs drive it

The task

The goal	Predict a continuous value — monthly revenue — from features like size, location, and marketing spend.
Two payoffs	A prediction for planning, and interpretable coefficients showing which drivers matter and by how much.
Why start here	Linear regression is the most interpretable model. Master reading it before reaching for black boxes.
Builds on S10	Same pipeline, split, and evaluation. Only the model and the metric (RMSE, R^2) change.

Source: DS Course, Session 11, 2026

02

PART 02

Linear Regression

Linear regression fits the straight line — or plane — that best predicts the target, and its coefficients are a readable story of what drives the outcome.

Linear regression: fit the best line, then read the coefficients

The line of best fit

OLS

Ordinary least squares finds the coefficients that minimize squared error — the line closest to all the points.

Coefficients

Each is the predicted change in the target per one-unit change in that feature, holding others fixed. Readable.

Intercept

The predicted value when every feature is zero — the baseline the coefficients adjust from.

PYTHON · OLS

```
from sklearn.linear_model import LinearRegression

ols = LinearRegression().fit(X_train, y_train)

# read the drivers
for name, coef in zip(features, ols.coef_):
    print(f'{name:20s} {coef:10.2f}')

ols.score(X_test, y_test)   # R^2
```

A coefficient = effect on the target, per unit of the feature.

03

PART 03

Reading the Model

A fitted model isn't trustworthy until you've checked how well it fits, whether its errors are healthy, and whether its features step on each other.

Don't trust a regression until you've read R², the residuals, and the correlations

Diagnose the fit

R²	The share of variance the model explains, 0 to 1. Higher is better — but a high R² on training alone can be overfitting.
Residuals	The errors. Plot them — they should look like random noise. A pattern means the model is missing something.
Multicollinearity	When features are highly correlated, coefficients become unstable and unreadable. Check correlations first.
RMSE	Average error in the target's own units — '฿42,000 off'. More intuitive to a stakeholder than R².

Source: DS Course, Session 11, 2026

04

PART 04

Why Regularization

Add enough features and a model fits the training data perfectly — and predicts the future terribly. Regularization is the cure for that overfitting.

Overfitting: a model that memorizes the past and fails on the future

The bias–variance trade-off

Overfitting	Too many features or too much flexibility and the model learns noise as if it were signal. Great train, poor test.
The trade-off	Too simple underfits (high bias); too complex overfits (high variance). The skill is finding the balance.
The tell	Training score far above test score. The gap is the overfitting, made visible by your held-out set.
The cure	Penalize large coefficients so the model can't chase every wiggle. That penalty is regularization.

Source: DS Course, Session 11, 2026

05

PART 05

Ridge & Lasso

Two regularizers with different gifts: Ridge shrinks every coefficient toward zero; Lasso drives some to exactly zero, selecting features for you.

Ridge and Lasso: shrink coefficients, and let Lasso select features

Two penalties

Ridge (L2)

Shrinks all coefficients toward zero, never quite reaching it. Stabilizes the model and tames multicollinearity.

Lasso (L1)

Pushes weak coefficients to exactly zero — automatic feature selection. The survivors are your important drivers.

Choosing alpha

Both have a strength knob, alpha. Cross-validate to pick it — RidgeCV and LassoCV do the search for you.

PYTHON · RIDGE & LASSO

```
from sklearn.linear_model import RidgeCV, LassoCV

ridge = RidgeCV(alphas=[0.1, 1, 10]).fit(
    X_train, y_train)
lasso = LassoCV(cv=5).fit(X_train, y_train)

# features Lasso kept (non-zero)
[f for f, c in zip(features, lasso.coef_) if c != 0]
```

Ridge stabilizes; Lasso selects. Cross-validate alpha.

06

PART 06

Workshop Block

Fit OLS and Ridge on your data, cross-validate, and report how much the regularized model improves out-of-sample error.

By the end of this block, every student has compared OLS to Ridge and reported the gain

Practice · 15 Minutes

Fit OLS	Baseline linear regression in your pipeline. Record the cross-validated RMSE.
Fit RidgeCV	Same features, with L2 regularization and a cross-validated alpha.
Compare	Report both CV RMSE numbers. Did Ridge improve out-of-sample error? By how much?
Interpret	Which coefficients shrank most? What does that say about which drivers were overfit? Your deliverable.

Source: Workshop, Session 11, 2026

KEY TAKEAWAY

Fit the training data; predict the future. Regularization protects the second.